
# DBMS_LAB

Lab-1 practice

Lab_3

Lab_4

Practice For MID

## ✅ 1. Create Database

Use this command to make a new database. It stores tables and all other data objects.

```
CREATE DATABASE lab_1;
```

---

## ✅ 2. Create Table

A table stores your data in rows and columns. You must be inside a database to create it.

```
CREATE TABLE students (
    id INT,
    name VARCHAR(50),
    age INT
);
```

---

## ✅ 3. Insert Multiple Values

You can add multiple rows at once using a comma-separated list of value sets.

```
INSERT INTO students (id, name, age)
VALUES
    (1, 'Araf', 22),
    (2, 'Salman', 25),
    (3, 'Partha', 28);
```

---

## ✅ 4. Show the Data

Use `SELECT` to display all rows from a table.

```
SELECT * FROM students;
```

---

## ✅ 5. Drop (Delete) Table & Database

This removes the entire table with all its data. Be careful—cannot be undone.

```
DROP TABLE students;
DROP DATABASE Lab_2;
```

---

## ✅ 6. DROP IF EXISTS

This safely drops a table **only if it exists**, preventing errors.

```
DROP TABLE IF EXISTS students;
```

---

## ✅ 7. CREATE IF NOT EXISTS

Creates a database or table **only if it doesn't already exist**.

```
CREATE DATABASE IF NOT EXISTS lab_1;

CREATE TABLE IF NOT EXISTS students (
    id INT,
    name VARCHAR(50),
    age INT
);
```

---

## ✅ 8. ALTER TABLE – Add Column

Use `ALTER TABLE` to modify an existing table structure by adding new columns.

```
ALTER TABLE students
ADD COLUMN email VARCHAR(100);
```

---

## ✅ 9. ALTER TABLE – Drop Column

Use `ALTER TABLE` to remove an existing column from a table. This will permanently delete the column and all its data.

```
ALTER TABLE students
DROP COLUMN email;
```

---

## ✅10. PRIMARY KEY (NOT NULL + UNIQUE)

A **Primary Key** uniquely identifies each row in a table. It automatically becomes **NOT NULL** and **UNIQUE**, meaning it cannot be empty and no two rows can have the same value.

```
CREATE TABLE IF NOT EXISTS students (
    id INT PRIMARY KEY,
    name VARCHAR(50),
    age INT
);
```

---

## ✅11. FOREIGN KEY (Linking Tables)

A **Foreign Key** creates a relationship between two tables by referencing the primary key of another table. This ensures data integrity by enforcing that the value in the foreign key column must exist in the referenced table.

```
CREATE TABLE IF NOT EXISTS courses (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(100)
);

CREATE TABLE IF NOT EXISTS enrollments (
    enrollment_id INT PRIMARY KEY,
    student_id INT,
    course_id INT,
    FOREIGN KEY (student_id) REFERENCES students(id),
    FOREIGN KEY (course_id) REFERENCES courses(course_id)
);
```

---

## ✅12. INSERT Values into Tables with Foreign Keys

When inserting data into a table with foreign keys, you must first insert the referenced data into the parent table, then insert into the child table.

```
-- First, insert data into the parent tables
INSERT INTO students (id, name, age)
VALUES (1, 'Araf', 22), (2, 'Salman', 25);

INSERT INTO courses (course_id, course_name)
VALUES (101, 'Database Systems'), (102, 'Data Structures');

-- Then insert into the child table with foreign key references
INSERT INTO enrollments (enrollment_id, student_id, course_id)
VALUES
   (1, 1, 101),  -- Araf enrolled in Database Systems
   (2, 1, 102),  -- Araf enrolled in Data Structures
   (3, 2, 101);  -- Salman enrolled in Database Systems
```

---

## ✅13. DEFAULT Values

A **DEFAULT value** automatically fills a column when no value is provided during insertion.

This helps avoid errors and ensures every row has a valid value even if the user does not supply one.

```
CREATE TABLE salary(
   id INT,
   salary FLOAT DEFAULT 25000.75
);

INSERT INTO salary(id)
VALUES (11);

--update the salary, cause by default it was 25000.75
```

```
UPDATE salary
SET salary = 30000.50
WHERE id = 11;
```

---

## ✅15. CHECK Constraint (Multiple Columns with AND)

A **CHECK constraint** can enforce conditions on **more than one column** using `AND` or `OR`.

### Example

```
CREATE TABLE Employee(
   id INT,
   age INT,
   city VARCHAR(50),
   CHECK (age >= 18 AND city = 'Dhaka')
);

-- ✅ Allowed
INSERT INTO Employee(id, age, city) VALUES (1, 25, 'Dhaka');

-- ❌ Error: age < 18 or city ≠ 'Dhaka'
INSERT INTO Employee(id, age, city) VALUES (2, 17, 'Dhaka');
INSERT INTO Employee(id, age, city) VALUES (3, 20, 'Chittagong');
```

---

## ✅ 16. Update Multiple Columns

Updates more than one column in a table using a single UPDATE statement.

```
UPDATE info
SET semester = 6,
```

```
    email = 'mkhan@gmail.com'
WHERE id = 1;
```

---

# ✅ 17. SELECT Operation

## 17(a). SELECT *

**One-line description:**

Selects **all columns** from a table.

**Code:**

```
SELECT * FROM info;
```

---

## 17(b). SELECT Specific Columns

**One-line description:**

Selects only the **columns you choose**.

**Code:**

```
SELECT name, age FROM info;
```

---

## 17(c). SELECT Specific Rows (using WHERE)

**One-line description:**

Shows only the rows that **match a condition**.

**Code:**

```
SELECT * FROM info WHERE age > 20;
```

---

# ✅ 18. SQL Operators

## 18(a). AND Operator

Used to select rows that satisfy **both** conditions.

```
SELECT * FROM info WHERE age > 18 AND city = 'Dhaka';
```

## 18(b). OR Operator

Used to select rows that satisfy **at least one** condition.

```
SELECT * FROM info WHERE city = 'Dhaka' OR city = 'Chittagong';
```

## 18(c). NOT Operator

Used to exclude rows that match a condition.

```
SELECT * FROM info WHERE NOT city = 'Dhaka';
```

## 18(d). IN Operator

Checks if a value matches **any value in a list**.

```
SELECT * FROM info WHERE city IN ('Dhaka', 'Sylhet', 'Rajshahi');
```

## 18(e). BETWEEN Operator

Checks if a value is **within a range**.

```
SELECT * FROM info WHERE age BETWEEN 18 AND 30;
```

## 18(f). LIKE Operator

Used for **pattern matching**.

```
SELECT * FROM info WHERE name LIKE 'S%';
--start with S
```

### 18(g). ANY Operator

Compares a value with **any value returned by a subquery**.

```
SELECT * FROM info WHERE age > ANY (SELECT age FROM temp);
```

### 18(h). ALL Operator

Compares a value with **all values returned by a subquery**.

```
SELECT * FROM info WHERE age > ALL (SELECT age FROM temp);
```

## ✅ 19. LIMIT (to show specific number of rows)

The **LIMIT** keyword is used to control how many rows are returned from a SELECT query.

### ✔️ Code Example:

```
SELECT * FROM info
LIMIT 5;
```

# ✅ 20. SORT Operation

### 20(a). Sort Ascending

Sorts the query result in **ascending order** (smallest to largest) based on a column.

**Code:**

```
SELECT * FROM info
ORDER BY age ASC;
```

### 20(b). Sort Descending

Sorts the query result in **descending order** (largest to smallest) based on a column.

**Code:**

```
SELECT * FROM info
ORDER BY age DESC;
```

# ✅ 21. AGGREGATE FUNCTIONS

### 21(a). COUNT()

Counts the **number of rows** in a table or matching a condition.

**Code:**

```
SELECT COUNT(*) FROM info;
```

### 21(b). MAX()

Finds the **maximum value** in a column.

**Code:**

```
SELECT MAX(age) FROM info;
```

### 21(c). MIN()

Finds the **minimum value** in a column.

**Code:**

```
SELECT MIN(age) FROM info;
```

---

### 21(d). SUM()

Calculates the **total sum** of a numeric column.

**Code:**

```
SELECT SUM(salary) FROM info;
```

---

### 21(e). AVG()

Calculates the **average value** of a numeric column.

**Code:**

```
SELECT AVG(age) FROM info;
```

---

## ✅ 22. GROUP BY

### 22(a). GROUP BY Basics

Groups rows that have the **same values** in a column so aggregate functions can be applied **per group**.

**Code:**

```
SELECT city, COUNT(*)
FROM info
GROUP BY city;
```

---

### 22(b). GROUP BY with Multiple Columns

Groups data based on **more than one column**.

**Code:**

```
SELECT city, gender, COUNT(*)
FROM info
GROUP BY city, gender;
```

---

### 22(c). GROUP BY with Aggregate Functions

Performs aggregate calculations **for each group**.

**Code:**

```
SELECT department, AVG(salary)
FROM info
GROUP BY department;
```

---

# ✅ 23. HAVING Clause

### 23(a). HAVING with GROUP BY

Filters **groups** created by `GROUP BY` (like WHERE, but for groups).

**Code:**

```
SELECT city, COUNT(*)
FROM info
GROUP BY city
HAVING COUNT(*) > 5;
```

---

### 23(b). HAVING with Aggregate Functions

Allows conditions using **aggregate functions** such as `SUM()`, `AVG()`, etc.

**Code:**

```
SELECT department, AVG(salary)
FROM info
GROUP BY department
HAVING AVG(salary) > 50000;
```

### 23(c). HAVING + WHERE Together

`WHERE` filters **rows**, `HAVING` filters **groups**.

**Code:**

```
SELECT city, SUM(sales)
FROM info
WHERE sales > 1000
GROUP BY city
HAVING SUM(sales) > 5000;
```

## ✅ 24. UPDATE Operation

### 24(a). Update a Single Column

Changes the value of **one column** in specific rows.

```
UPDATE info
SET age = 25
WHERE id = 3;
```

### 24(b). Update Multiple Columns

Updates **more than one column** at the same time.

```
UPDATE info
SET name = 'Rahim', age = 30
```

```
WHERE id = 5;
```

### 24(c). Update All Rows

Updates **every row** in the table.

```
UPDATE info
SET status = 'active';
```

## ✅ 25. DELETE Operation

### 25(a). Delete Specific Rows

Removes rows that match a given condition.

```
DELETE FROM info
WHERE id = 3;
```

### 25(b). Delete All Rows

Removes every row from the table but keeps the table structure.

```
DELETE FROM info;
```

### 25(c). Delete Using Multiple Conditions

Deletes rows based on more than one condition.

```
DELETE FROM info
WHERE city = 'Dhaka' AND age < 20;
```

## ✅ 26. FOREIGN KEY (FK)

## 26(a). Basic Foreign Key

Creates a relationship between two tables.

```
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    user_id INT,
    FOREIGN KEY (user_id) REFERENCES users(user_id)
);
```

## 26(b). Add Foreign Key to Existing Table

Adds an FK after the table is already created.

```
ALTER TABLE orders
ADD CONSTRAINT fk_user
FOREIGN KEY (user_id) REFERENCES users(user_id);
```

## 26(c). Foreign Key with ON DELETE CASCADE

Deletes child rows automatically when the parent row is deleted.

```
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    user_id INT,
    FOREIGN KEY (user_id) REFERENCES users(user_id)
    ON DELETE CASCADE
);
```

## 26(d). Drop a Foreign Key

Removes the foreign key constraint from a table.

```
ALTER TABLE orders
DROP CONSTRAINT fk_user;
```

# ✅ 27. SQL CONSTRAINTS

## 27(a). PRIMARY KEY (PK)

Uniquely identifies each row; cannot be NULL.

```
CREATE TABLE users (
    id INT PRIMARY KEY,
    name VARCHAR(50)
);
```

## 27(b). UNIQUE Constraint

Ensures all values in a column are unique.

```
CREATE TABLE users (
    email VARCHAR(100) UNIQUE,
    name VARCHAR(50)
);
```

## 27(c). CHECK Constraint

Sets a condition that values must satisfy.

```
CREATE TABLE employees (
    age INT,
    CHECK (age >= 18)
);
```

## 27(d). NOT NULL Constraint

Ensures the column **cannot** have a NULL value.

```
CREATE TABLE products (
    name VARCHAR(50) NOT NULL,
```

```
    price INT
);
```

# ✅ 28. FOREIGN KEY CASCADING

## 28(a). ON DELETE CASCADE

Automatically deletes child rows when the parent row is deleted.

```
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    user_id INT,
    FOREIGN KEY (user_id) REFERENCES users(user_id)
    ON DELETE CASCADE
);
```

## 28(b). ON UPDATE CASCADE

Automatically updates child rows when the parent key is updated.

```
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    user_id INT,
    FOREIGN KEY (user_id) REFERENCES users(user_id)
    ON UPDATE CASCADE
);
```

## 28(c). ON DELETE SET NULL

Sets the foreign key in child table to NULL when the parent row is deleted.

```
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    user_id INT,
```

```
    FOREIGN KEY (user_id) REFERENCES users(user_id)
    ON DELETE SET NULL
);
```

### 28(d). ON UPDATE SET NULL

Sets the foreign key in child table to NULL when the parent key is updated.

```
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    user_id INT,
    FOREIGN KEY (user_id) REFERENCES users(user_id)
    ON UPDATE SET NULL
);
```

## ✅ 29. TRUNCATE Operation

### 29(a). Basic TRUNCATE

Deletes **all rows** from a table quickly and resets storage, but keeps the table structure.

```
TRUNCATE TABLE info;
```

### 29(b). Difference from DELETE

`TRUNCATE` is **faster**, cannot have `WHERE`, and cannot be rolled back in some DBMS.

```
-- DELETE example for comparison
DELETE FROM info;
```

### 29(c). Usage Notes

Cannot truncate a table referenced by a **foreign key constraint** in some DBMS.

```
-- Will fail if foreign key exists
TRUNCATE TABLE orders;
```

---

# ✅ 30. JOINS

## 30(a). INNER JOIN

Returns rows that **have matching values** in both tables.

```
SELECT users.name, orders.order_id
FROM users
INNER JOIN orders
ON users.user_id = orders.user_id;
```

---

## 30(b). LEFT JOIN (or LEFT OUTER JOIN)

Returns **all rows from the left table**, and matched rows from the right table; NULL if no match.

```
SELECT users.name, orders.order_id
FROM users
LEFT JOIN orders
ON users.user_id = orders.user_id;
```

---

## 30(c). RIGHT JOIN (or RIGHT OUTER JOIN)

Returns **all rows from the right table**, and matched rows from the left table; NULL if no match.

```
SELECT users.name, orders.order_id
FROM users
```

```
RIGHT JOIN orders
ON users.user_id = orders.user_id;
```

---

### 30(d). FULL JOIN (or FULL OUTER JOIN)

Returns **all rows** when there is a match in either left or right table.

```
SELECT users.name, orders.order_id
FROM users
FULL OUTER JOIN orders
ON users.user_id = orders.user_id;
```

### 30(e). FULL JOIN using UNION

Simulates a **FULL OUTER JOIN** by combining LEFT and RIGHT JOIN results with `UNION`.

```
SELECT users.name, orders.order_id
FROM users
LEFT JOIN orders
ON users.user_id = orders.user_id

UNION

SELECT users.name, orders.order_id
FROM users
RIGHT JOIN orders
ON users.user_id = orders.user_id;
```

---

# ✅ 31. LEFT, RIGHT, and FULL EXCLUSIVE JOIN

### 31(a). LEFT EXCLUSIVE JOIN

Returns **rows from the left table** that **do not have matching rows** in the right table.

```
SELECT users.name
FROM users
LEFT JOIN orders
ON users.user_id = orders.user_id
WHERE orders.user_id IS NULL;
```

## 31(b). RIGHT EXCLUSIVE JOIN

Returns **rows from the right table** that **do not have matching rows** in the left table.

```
SELECT orders.order_id
FROM users
RIGHT JOIN orders
ON users.user_id = orders.user_id
WHERE users.user_id IS NULL;
```

## 31(c). FULL EXCLUSIVE JOIN

Returns **all rows from both tables** that **do not have matching rows** in the other table.

```
-- Combine left and right exclusive joins using UNION
SELECT users.name
FROM users
LEFT JOIN orders
ON users.user_id = orders.user_id
WHERE orders.user_id IS NULL

UNION

SELECT orders.order_id
FROM users
RIGHT JOIN orders
```

```
ON users.user_id = orders.user_id
WHERE users.user_id IS NULL;
```

---

# ✅ 32. UNION Operation

## 32(a). UNION

Combines the result of **two queries** and removes **duplicate rows**.

```
SELECT name FROM users
UNION
SELECT name FROM customers;
```

---

## 32(b). UNION ALL

Combines the result of **two queries** and **keeps duplicates**.

```
SELECT name FROM users
UNION ALL
SELECT name FROM customers;
```

---

## 32(c). UNION with ORDER BY

You can sort the combined results.

```
SELECT name FROM users
UNION
SELECT name FROM customers
ORDER BY name ASC;
```

---

# ✅ 33. SUBQUERY

## 33(a). Subquery in WHERE Clause

Uses a query inside **WHERE** to filter rows.

```
SELECT name, age
FROM employees
WHERE department_id = (
    SELECT department_id
    FROM departments
    WHERE name = 'Sales'
);
```

---

## 33(b). Subquery in FROM Clause

Treats a subquery as a **temporary table**.

```
SELECT dept, AVG(salary) AS avg_salary
FROM (
    SELECT department_id AS dept, salary
    FROM employees
) AS emp_dept
GROUP BY dept;
```

---

## 33(c). Subquery in SELECT Clause

Calculates a **value for each row** using a subquery.

```
SELECT name,
       (SELECT COUNT(*)
        FROM orders
        WHERE orders.user_id = users.user_id) AS order_count
FROM users;
```

---

## 33(d). Correlated Subquery

Subquery **depends on the outer query** for its values.

```
SELECT name, salary
FROM employees e1
WHERE salary > (
    SELECT AVG(salary)
    FROM employees e2
    WHERE e1.department_id = e2.department_id
);
```

---

# ✅ 34. VIEWS in MySQL

## 34(a). Create a Simple View

Creates a virtual table based on a query.

```
CREATE VIEW employee_view AS
SELECT name, department_id, salary
FROM employees
WHERE salary > 50000;
```

---

## 34(b). Query a View

Use the view like a normal table.

```
SELECT * FROM employee_view;
```

---

## 34(c). Update Through a View

You can update rows through a view if it maps to a **single table**.

```
UPDATE employee_view
SET salary = salary + 5000
WHERE department_id = 2;
```

---

## 34(d). Drop a View

Removes the view from the database.

```
DROP VIEW employee_view;
```

---